**Course:** Full Stack Web Development
**Course Code:** 23CM4121
**Task No.:** 2
**Student Name:** M. Vanisree
**Roll No.:** A24126552125
**Date:** 25-08-2026

## Title

Student Profile Using JavaScript

---

## Aim

To create a Student Profile application using JavaScript classes, objects, DOM manipulation, and event handling. The application accepts student details from the user and dynamically displays the student profile on the webpage when the button is clicked.

---

## OBJECTIVE

The objectives of this experiment are:

- To understand JavaScript classes and objects.
- To create and use a constructor in a JavaScript class.
- To retrieve HTML elements using getElementById().
- To handle user actions using addEventListener().
- To create objects dynamically using JavaScript classes.
- To manipulate webpage content using the DOM.
- To create HTML elements dynamically using createElement().
- To modify element content using textContent.
- To add dynamically created elements using appendChild().
- To display student information dynamically on a webpage.

---

## THEORY

JavaScript is a programming language used to make webpages interactive and dynamic. The Document Object Model (DOM) allows JavaScript to access and modify HTML elements and their content.

In this experiment, a Student Profile application is developed using JavaScript. The application accepts the student's Name, Roll Number, Department, and CGPA through input fields.

A JavaScript Student class is created with a constructor that initializes these student properties. When the user clicks the Create Profile button, the values entered in the input fields are retrieved using getElementById().

A new object of the Student class is then created using the entered values. DOM manipulation is used to dynamically create heading and paragraph elements to display the student's profile.

The following JavaScript concepts are used:

- Classes and Objects – Used to represent student information.
- Constructor – Initializes the properties of a Student object.
- getElementById() – Selects HTML elements using their IDs.
- addEventListener() – Handles the button click event.
- createElement() – Creates new HTML elements dynamically.
- textContent – Adds text content to dynamically created elements.
- appendChild() – Adds the created elements to the webpage.

The basic flow of the application is:

User Input → Button Click → JavaScript Event → Student Object → DOM Manipulation → Student Profile Display

---

## ALGORITHM / PROCEDURE

1. Create an HTML file containing input fields for student details.
2. Add fields for Name, Roll Number, Department, and CGPA.
3. Add a Create Profile button.
4. Create a container to display the generated student profile.
5. Create a CSS file to style the webpage.
6. Create a JavaScript file and define a Student class.
7. Define a constructor with Name, Roll Number, Department, and CGPA parameters.
8. Store the values in the properties of the Student object.
9. Select the Create Profile button using getElementById().

10. Add a click event using addEventListener().
11. Retrieve the values entered by the user from the input fields.
12. Create a new object of the Student class.
13. Select the profile display container.
14. Clear any previously displayed profile information.
15. Create heading and paragraph elements using createElement().
16. Add student details using textContent.
17. Add the created elements to the profile container using appendChild().
18. Open the HTML file in a web browser.
19. Enter student details and click the Create Profile button.
20. Verify that the student profile is displayed correctly.
21. Perform multiple test cases and verify the output.

---

## Program

## HTML Code:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <link rel="stylesheet" href="./style.css">
</head>
<body>
    <h1>Student details</h1>
    <div class="container">
        <label for="name">Name: </label>
        <input type="text" placeholder="Enter your Name: " id="name">

        <br><br>
        <label for="number">Roll Number: </label>
        <input type="text" placeholder="Enter your Roll Number: " id="number">

        <br><br>
        <label for="dept">Department: </label>
        <input type="text" placeholder="Enter your Department Name: " id="dept">

        <br><br>
        <label for="cgpa">CGPA: </label>
        <input type="text" placeholder="Enter your CGPA: " id="cgpa">

        <br><br>

        <button type="submit" id="createbtn">Create Profile</button>

        <div id="display-profile"></div>
    </div>
    <script src="./script.js"></script>
</body>
</html>
```

## CSS Code:

```css
body{
    background-color: beige;
    display: flex;
    flex-direction: column;
    margin: 0;
    justify-content: center;
    align-items: center;
    min-height: 100vh;
}
.container{
    background-color: aqua;
    display: flex;
    flex-direction: column;
    width: 550px;
    padding: 20px;
    box-shadow: 10px 20px 30px 40px rgba(0, 0, 0, 0.15);
    border-radius: 8px;
}
#name , #number, #dept, #cgpa{
    width: 500px;
    border-radius: 6px;
    border: none;
    background-color: azure;
}
button{
    width: 300px;
    margin-left: 200px;
    border-radius: 5px;
    border: none;
    padding: 10px;
    width: 200px;
    color: blue;
}
button:hover {
    background-color: darkblue;
}

#display-profile {
    background-color: white;
    padding: 20px;
    margin-top: 20px;
    border-radius: 10px;
    text-align: left;
    box-shadow: 0px 2px 8px gray;
}

#display-profile h2 {
    text-align: center;
}
```

## JavaScript Code:

```javascript
class Student{
    constructor(name, rollnumber, department, cgpa){
        this.name = name;
        this.rollnumber = rollnumber;
        this.dept = department;
        this.cgpa = cgpa;
    }
}

const createBtn = document.getElementById("createbtn");

createBtn.addEventListener("click", function(){
    const name = document.getElementById("name").value;
    const rollnumber = document.getElementById("number").value;
    const department = document.getElementById("dept").value;
    const cgpa = document.getElementById("cgpa").value;

    const student = new Student(
        name,
        rollnumber,
        department,
        cgpa
    );

    const profile =document.getElementById("display-profile");

    profile.innerHTML = "";

    const heading = document.createElement("h2");
    heading.textContent = "Student Profile";

    const namePara = document.createElement("p");
    namePara.textContent = "Name: " + student.name;

    const rollPara = document.createElement("p");
    rollPara.textContent = "Roll Number: " + student.rollnumber;

    const deptPara = document.createElement("p");
    deptPara.textContent = "Department: " + student.dept;

    const cgpaPara = document.createElement("p");
    cgpaPara.textContent = "CGPA: " + student.cgpa;

    profile.appendChild(heading);
    profile.appendChild(namePara);
    profile.appendChild(rollPara);
    profile.appendChild(deptPara);
    profile.appendChild(cgpaPara);
})
```

---

## CODE EXPLANATION

| Code | Explanation |
|---|---|
| class Student | Defines a JavaScript class for storing student information. |
| constructor() | Initializes the properties of a Student object. |
| this.name | Stores the student's name. |
| this.rollnumber | Stores the student's roll number. |

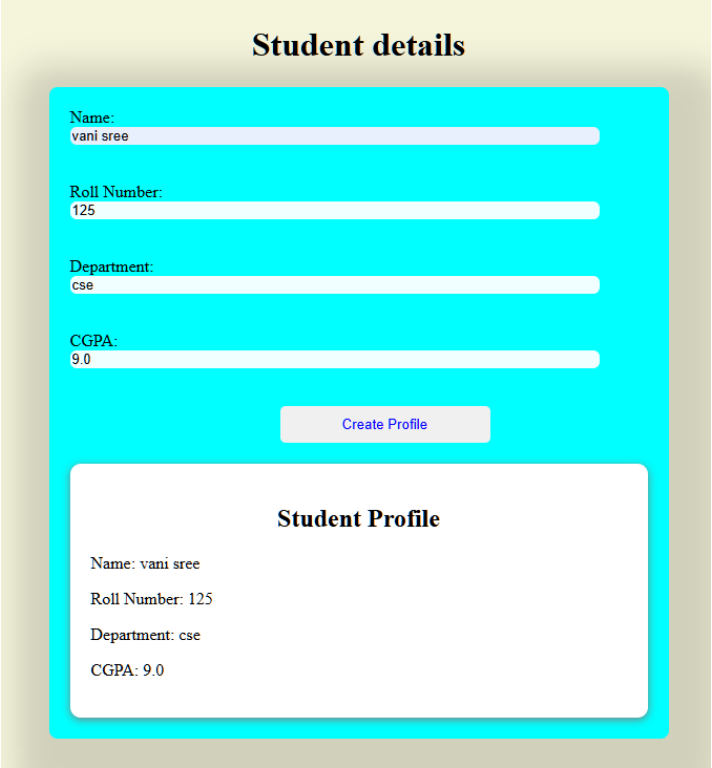
| | |
|---|---|
| this.dept | Stores the student's department. |
| this.cgpa | Stores the student's CGPA. |
| getElementById() | Selects an HTML element using its ID. |
| .value | Retrieves the value entered in an input field. |
| new Student() | Creates a new object of the Student class. |
| addEventListener() | Handles the click event of the button. |
| createElement() | Dynamically creates a new HTML element. |
| textContent | Sets the text content of an HTML element. |
| innerHTML = "" | Clears previously displayed profile content. |
| appendChild() | Adds dynamically created elements to the profile container. |
| button:hover | Changes the button appearance when the pointer is placed over it. |

---

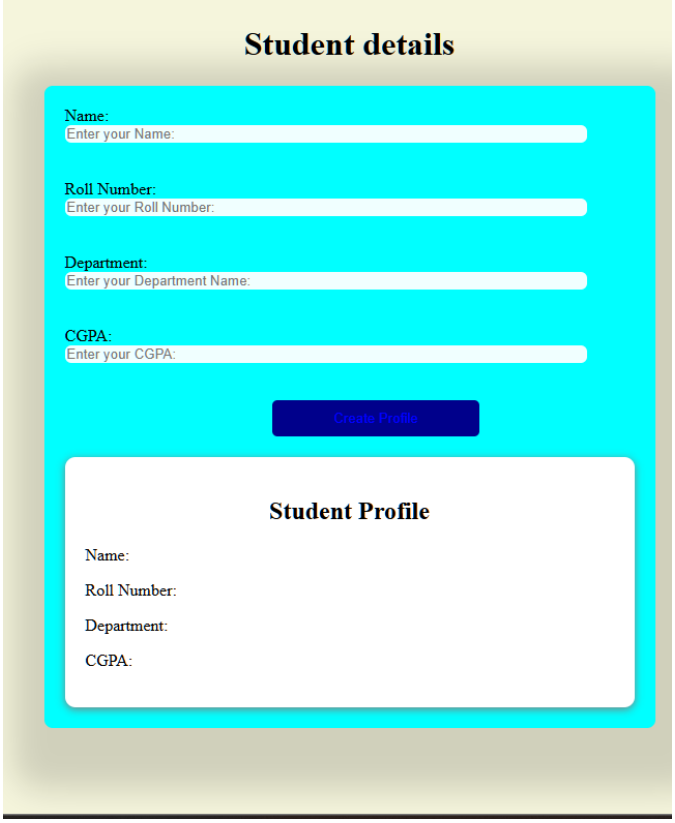
## TEST CASES:

| TC ID | Test Input | Expected Output | Actual Output | Status |
|---|---|---|---|---|
| **TC01** | Valid student details | Profile displayed with all details | Profile displayed correctly | Pass |
| **TC02** | Different valid student details | New profile displayed with updated details | Updated profile displayed | Pass |
| **TC03** | CGPA = 10.0 | CGPA should be displayed correctly | CGPA displayed correctly | Pass |
| **TC04** | Empty input fields | Empty input fields | Empty input fields | Empty input fields |

# TC OUTPUT 01 :

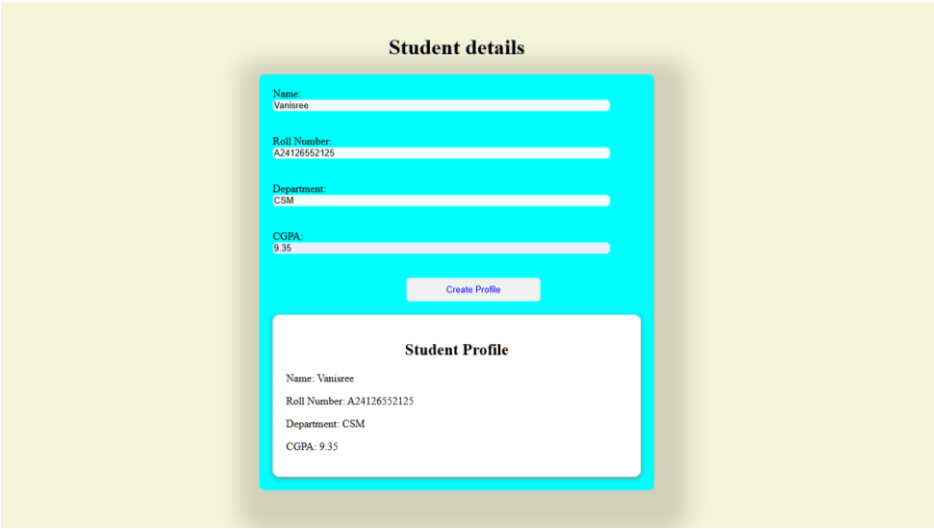



# TC OUTPUT 04:



## Output

When the user enters the student details and clicks the Create Profile button, the student profile is dynamically displayed on the webpage with the Name, Roll Number, Department, and CGPA.




---

## RESULT

Thus, a **Student Profile application using JavaScript** was successfully created. JavaScript classes and objects, constructors, DOM manipulation, event handling, getElementById(), addEventListener(), createElement(), textContent, and appendChild() were implemented successfully. The student details were accepted from the user and dynamically displayed as a profile on the webpage.